UCL CASA Coursework: Data Science for Spatial Systems (CASA0006).

Big Data Analytics For Road Accident Prediction : A case study of London.

- About this document:

This is the assignment for the UCL CASA module Data Science for Spatial Systems (CASA0006)

This is a self-contained Jupyter notebook that includes structure requirements, introduction, literature review, results, discussion, and conclusion, with embedded code and a bibliography..

Acronyms:

- RF = Random Forest
- ML = Machine Learning
- XGBoost = Gradient Boosting
- CASA = Centre for Advanced Spatial Analysis
- UCL = University College London
- OS = Operating system

Application and Goals

Using this dataset, we will develop two models to predict accident occurrence based on Random Forest and XGBoost methodology.

The processed dataset will facilitate predictive modeling tasks aimed at achieving the following goals:

- **Predict Accident Likelihood**: Forecast the likelihood of accidents occurrence based on contributing factors such as weather, timing, type of road and road design.
- **Feature important**: predict which variable is a major factor in accident prediction in London.
- **Analyze the model performance**:

This analysis prepares the dataset for RF and XGBoost tasks, with the aim of deriving insights into accident prediction and enhancing road safety models for London City.

Requirements

[go back to the top]

Preparation

- Github link *[Github]*
- Number of words: 1486
- Runtime: *** hours (*Memory 8 GB, CPU Intel i5-10700 CPU @2.50 *)
- Coding environment: SDS2024 powered by Anaconda, run on VS Code hosted on Github for reproducibility
- License: this notebook is made available under the Creative Commons Attribution license (or other license that you like).

First step is to import the relevant libraries, which will allow us to perform a supervised machine learning workflow for road accident prediction in London.

- `pandas` for data import and handling;
- `matplotlib` ; for visualization
- `numpy` ; for manipulation numbers
- `sklearn` ; for Machine learning
- `xgboost` for XGBoost models.
- `rfpimp` for permutation of feature importance.

- **Additional library**
- **SMOTE**: for balancing data imbalance
- !pip install **pandoc** : for pdf saving pdf
- **TeX Live** and **MiKTeX**: for generating pdf from jupyter notebook on windows OS

Let's run the script below to get started.

In [69]:

```
# Import necessary libraries

# Data manipulation
import pandas as pd
import numpy as np
import geopandas as gpd
import os
import zipfile
import requests
import io

# Visualization
import matplotlib.pyplot as plt
import seaborn as sns

# Machine learning & preprocessing
from sklearn.model_selection import train_test_split, GridSearchCV, cross_val_score
from sklearn.preprocessing import LabelEncoder, StandardScaler
from sklearn.impute import SimpleImputer
from imblearn.over_sampling import SMOTE

# Model selection
from xgboost import XGBClassifier
from sklearn.ensemble import RandomForestClassifier

# Evaluation metrics
from sklearn.metrics import (
    classification_report,
    confusion_matrix,
    accuracy_score,
    f1_score,
    precision_score,
    recall_score,
)

# Function to print package versions
import importlib

def get_version(package_name):
    try:
        pkg = importlib.import_module(package_name)
        return pkg.__version__
    except AttributeError:
        return "Version attribute not found"
    except ModuleNotFoundError:
        return "Package not installed"

# List of packages to check versions
packages = [
    "pandas", "numpy", "geopandas", "matplotlib", "seaborn",
    "sklearn", "xgboost", "imblearn", "requests"
]

# Print versions
```

```python
for pkg in packages:
    print(f"{pkg.capitalize()} Version: {get_version(pkg)}")
```

```
Pandas Version: 2.2.3
Numpy Version: 2.0.0
Geopandas Version: 1.0.1
Matplotlib Version: 3.9.2
Seaborn Version: 0.13.2
Sklearn Version: 1.6.1
Xgboost Version: 3.0.0
Imblearn Version: 0.13.0
Requests Version: 2.32.3
```

Table of contents

2.0 Introduction

[go back to the top]

Introduction

Recent studies project that by 2030, traffic accidents will rank as the fifth leading cause of death worldwide (Venkat et al., 2020). Each year, approximately 1.2 million fatalities are linked to road accidents, influenced by factors such as weather conditions, time of day, accident location, traffic volume, and road design (Anjuman et al., 2020). Beyond the human toll, these incidents impose significant economic and social costs, including injuries, loss of productivity, and damage to infrastructure (Pourroostaei Ardakani et al., 2023).

The rapid emergence of autonomous vehicles introduces new complexities, raising concerns about how accident patterns may evolve in the near future. Against this backdrop, applying machine learning methods to publicly available large-scale datasets such as those collected across London offers a valuable opportunity to better understand, predict, and potentially mitigate accident risks.

Consequently, developing data-driven insights through machine learning analytics can provide a promising approach to inform safer transport systems and support evidence-based policymaking.

2.1 Research questions

[go back to the top]

Prediction: How effectively can supervised machine learning models predict the likelihood of road accidents based on weather conditions, time of day, traffic volume, road design, and accident location?

Feature Importance: Which environmental, temporal, and infrastructural factors most strongly influence the occurrence of road traffic accidents in London?

Model Performance: How well do the selected supervised machine learning models perform in terms of predictive accuracy and reliability?

3.0 Literature review

The development of accident prediction models has been extensively studied as part of efforts to enhance road safety. Researchers have sought to build models that are both accurate and practically applicable, often incorporating explanatory variables to better understand the underlying causes of accidents (Yannis et al., 2017).

In Iran, Khosravi et al. (2024) highlighted the importance of accurate accident prediction, applying both supervised and unsupervised machine learning algorithms to predict accident severity. Similarly, in Ethiopia, Beshah et al. (2011) employed classification algorithms to predict crash severity using variables such as road conditions and driver behavior, achieving high levels of accuracy. In the UK, Pourroostaei Ardakani et al. (2023) demonstrated the effectiveness of supervised machine learning models in accident prediction, emphasizing the critical role of environmental and infrastructural conditions.

Collectively, these studies underscore the value of leveraging big data and advanced machine learning techniques to improve accident prediction models. They highlight not only the potential for enhanced accuracy but also the importance of contextual variables ranging from environmental factors to driver behavior in shaping predictive outcomes.

Building on this body of work, the present research applies similar machine learning methodologies within the specific context of London. By focusing on localized accident data, this study aims to generate insights that are both practically relevant and complementary to existing literature, further validating the role of machine learning in advancing road safety.

4.0 Data

[go back to the top]

[2023 London Road Accident Dataset and Lsoa shapefile.]

Variables	Type	Description	Notes
accident_year	Numeric	Year in which the accident occurred.	
longitude	Numeric	Longitude coordinate where the accident occurred.	Handle missing values.
latitude	Numeric	Latitude coordinate where the accident occurred.	.
accident_severity	Categorical	Severity level of the accident (e.g., minor, serious, fatal).	Target variable for prediction.
number_of_vehicles	Numeric	Number of vehicles involved in the accident.	
number_of_casualties	Numeric	Number of casualties resulting from the accident.	
date	Categorical	Date of the accident in DD/MM/YYYY format.	Used for for temporal analysis.
day_of_week	Categorical	Day of the week when the accident occurred (e.g., Monday = 2).	Encoded as an integer (1 = Sunday).
time	Categorical	Time of the accident (HH:MM format).	Useful for time bucket creation.
road_type	Categorical	Type of road where the accident occurred (e.g., roundabout, dual carriageway).	Encoded as an integer.

Variables	Type	Description	Notes
speed_limit	Numeric	Speed limit of the road at the accident location.	
junction_detail	Categorical	Details about the junction (e.g., roundabout, crossroads).	Encoded as an integer.
junction_control	Categorical	Type of control at the junction (e.g., traffic signal, stop sign).	Encoded as an integer.
light_conditions	Categorical	Light conditions during the accident (e.g., daylight, darkness).	Encoded as an integer.
weather_conditions	Categorical	Weather conditions during the accident (e.g., rain, fog).	Encoded as an integer.
road_surface_conditions	Categorical	Condition of the road surface during the accident (e.g., dry, wet).	Encoded as an integer.
urban_or_rural_area	Categorical	Indicates whether the accident occurred in an urban or rural area.	Binary encoding (1 = urban, 2 = rural).
lsoa_of_accident_location	Categorical	Lower Layer Super Output Area where the accident occurred.	for spatial analysis if word cout allow.

We will use the Pandas package to load and explore these datasets:

1. **Load Road Accident Dataset**: Import the dataset and convert columns like `date` and `time` into formats that are suitable for ML analysis.
2. **Inspect the Data**: Explore the structure .
3. **Aggregate Features**:
4. **Create New Features**: Extract temporal and environmental features such as `season`, `day_of_week`, and `is_weekend`

In [ ]:
```
# We will load our CSV from GitHub
def load_csv(url, parse_dates=None):
    try:
        return pd.read_csv(url, parse_dates=parse_dates, dayfirst=True, low_memory=
    except Exception as e:
        print(f"CSV load error: {e}")
        return None

# Load zipped shapefile from GitHub
def load_shapefile(zip_url, extract_to, shapefile_name):
    try:
        with zipfile.ZipFile(io.BytesIO(requests.get(zip_url).content)) as z:
            z.extractall(extract_to)
        for root, _, files in os.walk(extract_to):
            if shapefile_name in files:
                return gpd.read_file(os.path.join(root, shapefile_name))
        print(f"Shapefile '{shapefile_name}' not found.")
```

```
    except Exception as e:
        print(f"Shapefile load error: {e}")
    return None

# URLs and filenames
road_accident_url = "https://raw.githubusercontent.com/Idrisbaba/Accident_Predictio
lsoa_zip_url = "https://github.com/Idrisbaba/Accident_Prediction/raw/main/Data/Shap
road_zip_url = "https://github.com/Idrisbaba/Accident_Prediction/raw/main/Data/Shap

lsoa_folder = "temp_lsoa"
road_folder = "temp_roads"
lsoa_shapefile = "LSOA_2011_London_gen_MHW.shp"
road_shapefile = "Major_Road_Network_2018_Open_Roads.shp"

# Load datasets
df_accidents = load_csv(road_accident_url, parse_dates=['date'])
gdf_lsoa = load_shapefile(lsoa_zip_url, lsoa_folder, lsoa_shapefile)
gdf_roads = load_shapefile(road_zip_url, road_folder, road_shapefile)

# Preview
if df_accidents is not None:
    print("\nRoad Accident Data Preview:")
    print(df_accidents.head())

if gdf_lsoa is not None:
    print("\nLSOA Shapefile Preview:")
    print(gdf_lsoa.head())

if gdf_roads is not None:
    print("\nMajor Road Network Preview:")
    print(gdf_roads.head())
```

```
Road Accident Data Preview:
  accident_index  accident_year accident_reference  location_easting_osgr  \
0  2023010419171           2023          010419171               525060.0
1  2023010419183           2023          010419183               535463.0
2  2023010419189           2023          010419189               508702.0
3  2023010419191           2023          010419191               520341.0
4  2023010419192           2023          010419192               527255.0

   location_northing_osgr  longitude   latitude  police_force  \
0                170416.0  -0.202878  51.418974             1
1                198745.0  -0.042464  51.671155             1
2                177696.0  -0.435789  51.487777             1
3                190175.0  -0.263972  51.597575             1
4                176963.0  -0.168976  51.477324             1

   accident_severity  number_of_vehicles  ...  light_conditions  \
0                  3                   1  ...                 4
1                  3                   3  ...                 4
2                  3                   2  ...                 4
3                  3                   2  ...                 4
4                  3                   2  ...                 4

  weather_conditions  road_surface_conditions special_conditions_at_site  \
0                  8                        2                          0
1                  1                        1                          0
2                  1                        1                          0
3                  9                        1                          0
4                  1                        1                          0

   carriageway_hazards urban_or_rural_area  \
0                    0                   1
1                    0                   1
2                    0                   1
3                    0                   1
4                    0                   1

  did_police_officer_attend_scene_of_accident  trunk_road_flag  \
0                                           1                2
1                                           1                2
2                                           1                2
3                                           1                2
4                                           1                2

   lsoa_of_accident_location  enhanced_severity_collision
0                  E01003383                           -1
1                  E01001547                           -1
2                  E01002448                           -1
3                  E01000129                           -1
4                  E01004583                           -1

[5 rows x 37 columns]

LSOA Shapefile Preview:
    LSOA11CD                  LSOA11NM   MSOA11CD                 MSOA11NM  \
0  E01000001        City of London 001A  E02000001        City of London 001
1  E01000002        City of London 001B  E02000001        City of London 001
```

```
2  E01000003        City of London 001C  E02000001        City of London 001
3  E01000005        City of London 001E  E02000001        City of London 001
4  E01000006  Barking and Dagenham 016A  E02000017  Barking and Dagenham 016

     LAD11CD               LAD11NM    RGN11CD RGN11NM  USUALRES  HHOLDRES  \
0  E09000001        City of London  E12000007  London      1465      1465
1  E09000001        City of London  E12000007  London      1436      1436
2  E09000001        City of London  E12000007  London      1346      1250
3  E09000001        City of London  E12000007  London       985       985
4  E09000002  Barking and Dagenham  E12000007  London      1703      1699

   COMESTRES  POPDEN  HHOLDS  AVHHOLDSZ  \
0          0   112.9     876        1.7
1          0    62.9     830        1.7
2         96   227.7     817        1.5
3          0    52.0     467        2.1
4          4   116.2     543        3.1

                                             geometry
0  POLYGON ((532105.092 182011.23, 532162.491 181...
1  POLYGON ((532746.813 181786.891, 532671.688 18...
2  POLYGON ((532135.145 182198.119, 532158.25 182...
3  POLYGON ((533807.946 180767.77, 533649.063 180...
4  POLYGON ((545122.049 184314.931, 545271.917 18...

Major Road Network Preview:
   fid_                                    id  \
0   0.0  id83DAD742-902E-40A4-94F1-01014A477BD0
1   0.0  id14F16D2F-CF52-4AA7-BCA4-0B1A7A19D337
2   0.0  idA5563E2D-8D74-4834-93DE-095BF5E2F875
3   0.0  id2A013B71-961C-4FFA-9564-63212F981A19
4   0.0  id9658E036-E0D1-4A5B-A413-C892F3F7B169

                                endNode  \
0  id78290C31-7EAD-428E-80A4-3C5B0E8F7AD1
1  idF3EC06D8-F396-449D-972C-B0B37A448FBD
2  idEB4B7309-2F2D-4661-B55B-58F15758370A
3  idB226C011-36A8-4C3E-A16D-4F2DF5612118
4  id37DFB9F0-69DB-4FD2-A308-FF3FA9BA2CF6

                              startNode            roadNumber roadNameTO  \
0  id3500BDA4-9041-4E08-857E-51CCC47EBC1F  osgb4000000006807237       None
1  id78290C31-7EAD-428E-80A4-3C5B0E8F7AD1  osgb4000000006807237       None
2  idF3EC06D8-F396-449D-972C-B0B37A448FBD  osgb4000000006807237       None
3  idEB4B7309-2F2D-4661-B55B-58F15758370A  osgb4000000006807237       None
4  id94F4EEFC-DE24-4C11-B6FC-14AEF1A9CE32  osgb4000000006807237       None

   fictitious roadClassi roadFuncti           formOfWay  ...  trunkRoad  \
0         0.0     A Road     A Road  Single Carriageway  ...        0.0
1         0.0     A Road     A Road  Single Carriageway  ...        0.0
2         0.0     A Road     A Road  Single Carriageway  ...        0.0
3         0.0     A Road     A Road  Single Carriageway  ...        0.0
4         0.0     A Road     A Road  Single Carriageway  ...        0.0

  roadClas_1  name1  name1_lang  name2 name2_lang roadStruct   geom_Lengt  \
0        A68   None        None   None       None       None   597.894361
```

```
1        A68   None        None   None       None       None   619.288146
2        A68   None        None   None       None       None   475.472415
3        A68   None        None   None       None       None   367.208526
4        A68   None        None   None       None       None  2642.746596

                             identifier  \
0  83DAD742-902E-40A4-94F1-01014A477BD0
1  14F16D2F-CF52-4AA7-BCA4-0B1A7A19D337
2  A5563E2D-8D74-4834-93DE-095BF5E2F875
3  2A013B71-961C-4FFA-9564-63212F981A19
4  9658E036-E0D1-4A5B-A413-C892F3F7B169

                                          geometry
0  LINESTRING (375279 603208, 375425 603174, 3755...
1  LINESTRING (375867 603101, 375960 603084, 3760...
2  LINESTRING (376419 602834, 376630 602724.47, 3...
3  LINESTRING (376831 602598, 376958 602495, 3769...
4  LINESTRING (371309 604984, 371272 604992, 3710...

[5 rows x 24 columns]
```

In [95]:

```python
# Define columns to drop
columns_to_drop = [
    'accident_reference', 'accident_index', 'police_force', 'local_authority_distri
    'local_authority_ons_district', 'local_authority_highway', 'first_road_class',
    'pedestrian_crossing_physical_facilities', 'pedestrian_crossing_human_control',
    'second_road_number', 'first_road_number', 'carriageway_hazards',
    'special_conditions_at_site', 'trunk_road_flag', 'second_road_class',
    'did_police_officer_attend_scene_of_accident', 'enhanced_severity_collision'
]

# Retain coordinates for spatial joins, drop OSGR if lat/long present
if 'longitude' in df_accidents.columns and 'latitude' in df_accidents.columns:
    columns_to_drop += ['location_easting_osgr', 'location_northing_osgr']

# Drop only existing columns
df_accidents.drop(columns=[col for col in columns_to_drop if col in df_accidents.co

# Impute missing values (full dataset before split)
imputer = SimpleImputer(strategy="most_frequent")
df_accidents = pd.DataFrame(imputer.fit_transform(df_accidents), columns=df_acciden

# Display updated structure
print(f"\nUpdated dataset shape: {df_accidents.shape}")
print("Dataset info after dropping and imputation:")
print(df_accidents.info())
print(df_accidents.head())
```

```
Updated dataset shape: (104258, 18)
Dataset info after dropping and imputation:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 104258 entries, 0 to 104257
Data columns (total 18 columns):
 #   Column                     Non-Null Count   Dtype
---  ------                     --------------   -----
 0   accident_year              104258 non-null  object
 1   longitude                  104258 non-null  object
 2   latitude                   104258 non-null  object
 3   accident_severity          104258 non-null  object
 4   number_of_vehicles         104258 non-null  object
 5   number_of_casualties       104258 non-null  object
 6   date                       104258 non-null  datetime64[ns]
 7   day_of_week                104258 non-null  object
 8   time                       104258 non-null  object
 9   road_type                  104258 non-null  object
 10  speed_limit                104258 non-null  object
 11  junction_detail            104258 non-null  object
 12  junction_control           104258 non-null  object
 13  light_conditions           104258 non-null  object
 14  weather_conditions         104258 non-null  object
 15  road_surface_conditions    104258 non-null  object
 16  urban_or_rural_area        104258 non-null  object
 17  lsoa_of_accident_location  104258 non-null  object
dtypes: datetime64[ns](1), object(17)
memory usage: 14.3+ MB
None
  accident_year longitude   latitude accident_severity number_of_vehicles  \
0          2023 -0.202878  51.418974                 3                   1
1          2023 -0.042464  51.671155                 3                   3
2          2023 -0.435789  51.487777                 3                   2
3          2023 -0.263972  51.597575                 3                   2
4          2023 -0.168976  51.477324                 3                   2

  number_of_casualties       date day_of_week   time road_type speed_limit  \
0                    1 2023-01-01           1  01:24         2          20
1                    2 2023-01-01           1  02:25         6          30
2                    1 2023-01-01           1  03:50         1          30
3                    1 2023-01-01           1  02:13         6          30
4                    1 2023-01-01           1  01:42         6          30

  junction_detail junction_control light_conditions weather_conditions  \
0               9                4                4                  8
1               3                4                4                  1
2               1                4                4                  1
3               3                4                4                  9
4               8                4                4                  1

  road_surface_conditions urban_or_rural_area lsoa_of_accident_location
0                       2                   1                E01003383
1                       1                   1                E01001547
2                       1                   1                E01002448
3                       1                   1                E01000129
4                       1                   1                E01004583
```

In [96]:
```
# Drop columns that are in columns_to_drop and exist in df_accidents
df_accidents.drop(columns=[col for col in columns_to_drop if col in df_accidents.co

# Impute missing values in df_accidents
imputer = SimpleImputer(strategy="most_frequent")
df_accidents = pd.DataFrame(imputer.fit_transform(df_accidents), columns=df_acciden
```

In [73]:
```
# print a few rows of this dataset
df_accidents.head()
```

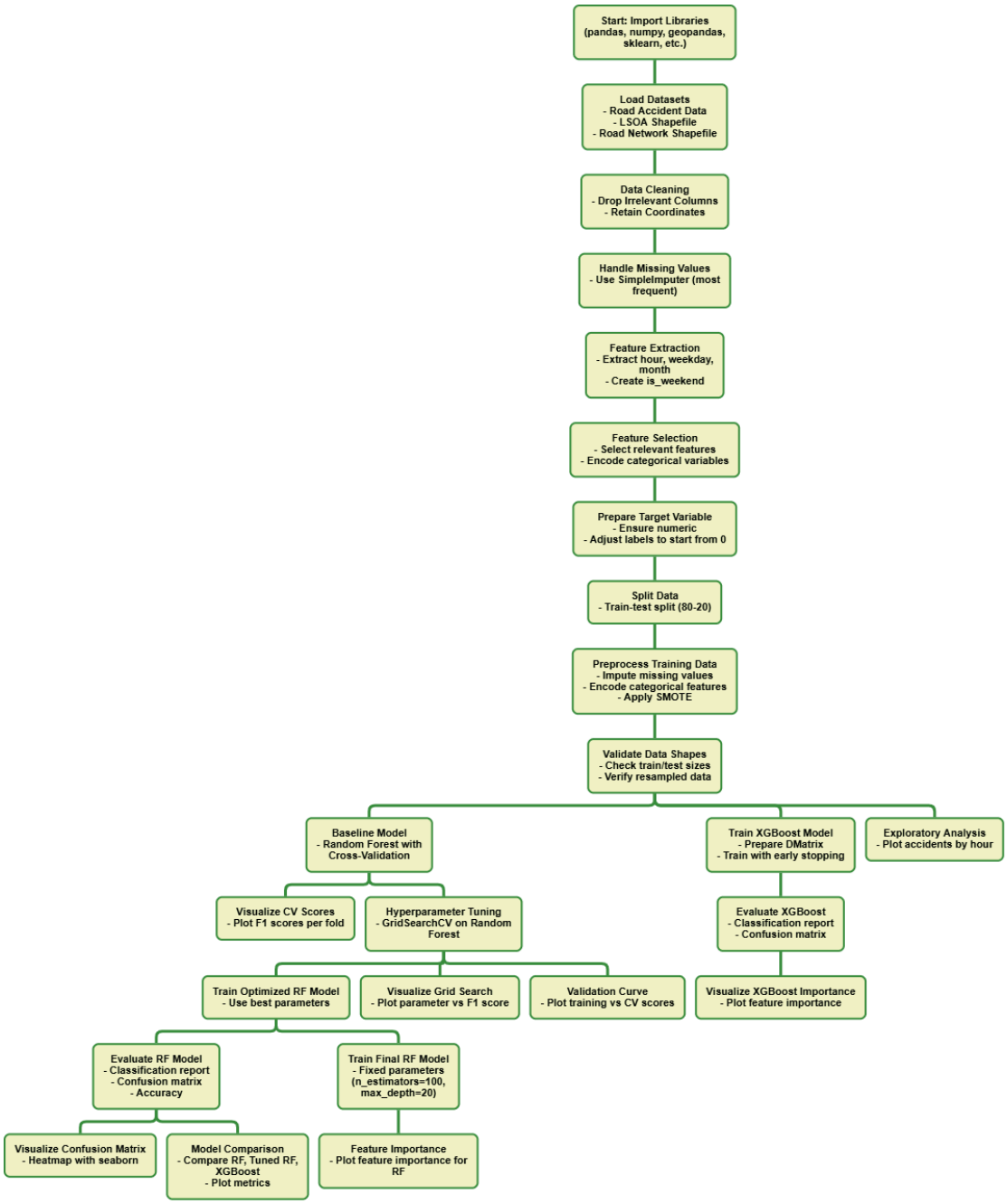
Out[73]:

	accident_year	longitude	latitude	accident_severity	number_of_vehicles	number_of_ca
0	2023	-0.202878	51.418974	3	1	
1	2023	-0.042464	51.671155	3	3	
2	2023	-0.435789	51.487777	3	2	
3	2023	-0.263972	51.597575	3	2	
4	2023	-0.168976	51.477324	3	2	

5.0 Methodology

[go back to the top]

In [74]:
```
import urllib.request
urllib.request.urlretrieve(
    "https://github.com/Idrisbaba/Accident_Prediction/raw/main/pictures/workflow.pn
    "workflow.png"
)

from IPython.display import Image, display
display(Image(filename="workflow.png"))
```



Model Flow Chart

5.1 Methodology

This study adopts a data-driven approach to predict and visualize the likelihood of road accidents by identifying and forecasting the contributing factors in London. As noted in the literature review, several machine learning approaches have been employed by researchers for classification and predictive modeling, which are increasingly recognized as effective tools for investigating accident occurrence (Takale et al., 2022). Combining clustering techniques, such as K-means, with supervised machine learning classification algorithms,

such as Random Forest, has demonstrated promising results in improving prediction accuracy (Yassin and Pooja, 2020). Therefore, this study leverages similar methodologies, including Random Forest and XGBoost, which have shown clear improvements over traditional machine learning techniques; these methods often achieve higher accuracy and greater robustness compared to single models (Morales and Escalante, 2022).

5.2 Limitations of Supervised Machine Learning: Machine learning methodologies are entirely dependent on the quality of the data; therefore, both data size and data quality are critical in the application of road accident prediction analysis (Khosravi et al., 2024). This reflects a classic "garbage in, garbage out" scenario, where inaccurate data leads to skewed and unreliable results because computer models learn from the input provided (Mitchell, 1997). Consequently, in accident prediction, the amount and quality of training data can significantly constrain model performance and predictive accuracy.

6.0 Results and discussion

[go back to the top]

In [129…

```
# Feature extraction from date and time
df_accidents['hour'] = df_accidents['time'].str[:2].astype(float)
df_accidents['weekday'] = df_accidents['date'].dt.weekday
df_accidents['month'] = df_accidents['date'].dt.month
df_accidents['is_weekend'] = df_accidents['weekday'] >= 5

# Feature selection
feature_cols = [
    'hour', 'weekday', 'month', 'is_weekend',
    'weather_conditions', 'light_conditions', 'road_surface_conditions',
    'road_type', 'junction_detail', 'junction_control',
    'speed_limit', 'urban_or_rural_area',
    'number_of_vehicles', 'number_of_casualties'
]

# Define input features and target
X = df_accidents[feature_cols].copy()
y = df_accidents['accident_severity']

# encode categorical columns
categorical_cols = X.select_dtypes(include='object').columns
if len(categorical_cols) > 0:
    label_encoder = LabelEncoder()
    for col in categorical_cols:
        X[col] = label_encoder.fit_transform(X[col].astype(str))

# Preview encoded features
print("Feature columns after encoding:")
print(X.head())
```

```
Feature columns after encoding:
   hour  weekday  month  is_weekend  weather_conditions  light_conditions  \
0   1.0        6      1        True                   7                 1
1   2.0        6      1        True                   0                 1
2   3.0        6      1        True                   0                 1
3   2.0        6      1        True                   8                 1
4   1.0        6      1        True                   0                 1

   road_surface_conditions  road_type  junction_detail  junction_control  \
0                        2          1                9                 4
1                        1          3                4                 4
2                        1          0                2                 4
3                        1          3                4                 4
4                        1          3                8                 4

   speed_limit  urban_or_rural_area  number_of_vehicles  number_of_casualties
0            0                    1                   0                     0
1            1                    1                   9                     9
2            1                    1                   8                     0
3            1                    1                   8                     0
4            1                    1                   8                     0
```

In [76]:
```
df_accidents.head()
```

Out[76]:

	accident_year	longitude	latitude	accident_severity	number_of_vehicles	number_of_ca
0	2023	-0.202878	51.418974	3	1	
1	2023	-0.042464	51.671155	3	3	
2	2023	-0.435789	51.487777	3	2	
3	2023	-0.263972	51.597575	3	2	
4	2023	-0.168976	51.477324	3	2	

5 rows × 22 columns

6.1 Data Preprocessing and Class Balancing for Road Accident Prediction

This step ensures that missing values are properly handled, target labels are corrected, categorical features are encoded, the dataset is balanced, preprocessing steps are validated, and the data is split into a training set.

In [97]:
```
# Step 1: Ensure the target variable is numeric and zero-based
y = y.astype(int)
if y.min() < 0:
```

```
    y = y - y.min()
y = y - y.min()  # Ensure zero-based encoding
print("Unique values in y:", np.unique(y))

# Step 2: Split the data
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta

# Step 3: Impute missing values
imputer = SimpleImputer(strategy="most_frequent")
X_train = pd.DataFrame(imputer.fit_transform(X_train), columns=X_train.columns)

# Step 4: Encode categorical features
categorical_cols = X_train.select_dtypes(include=['object']).columns
for col in categorical_cols:
    X_train[col] = LabelEncoder().fit_transform(X_train[col].astype(str))

# Step 5: Apply SMOTE
smote = SMOTE(random_state=42)
X_train_resampled, y_train_resampled = smote.fit_resample(X_train, y_train)
y_train_resampled = y_train_resampled - y_train_resampled.min()  # Zero-based

print("Unique values in y_train_resampled:", np.unique(y_train_resampled))
print("X_train_resampled shape:", X_train_resampled.shape)
print("Class distribution in y_train_resampled:\n", pd.Series(y_train_resampled).va
```

```
Unique values in y: [0 1 2]
Unique values in y_train_resampled: [0 1 2]
X_train_resampled shape: (190623, 14)
Class distribution in y_train_resampled:
 accident_severity
1    63541
2    63541
0    63541
Name: count, dtype: int64
```

Now let's validate the shape for our analysis

In [99]:
```
# Validate the shape of the original training and testing sets
print(f"Original Training set size: {X_train.shape}, Target size: {y_train.shape}")
print(f"Original Test set size: {X_test.shape}, Target size: {y_test.shape}")

# Validate the shape of the resampled data
print(f"Resampled Training set size: {X_train_resampled.shape}, Target size: {y_tra

# Ensure the number of samples in X_train_resampled matches y_train_resampled
assert X_train_resampled.shape[0] == y_train_resampled.shape[0], "Mismatch in sampl

# Ensure the number of features remains consistent after resampling
assert X_train_resampled.shape[1] == X_train.shape[1], "Mismatch in feature count a

# Output a summary of all shapes
print("Data shape validation passed. All dimensions match correctly.")
```

```
Original Training set size: (83406, 14), Target size: (83406,)
Original Test set size: (20852, 14), Target size: (20852,)
Resampled Training set size: (190623, 14), Target size: (190623,)
Data shape validation passed. All dimensions match correctly.
```

In [100…

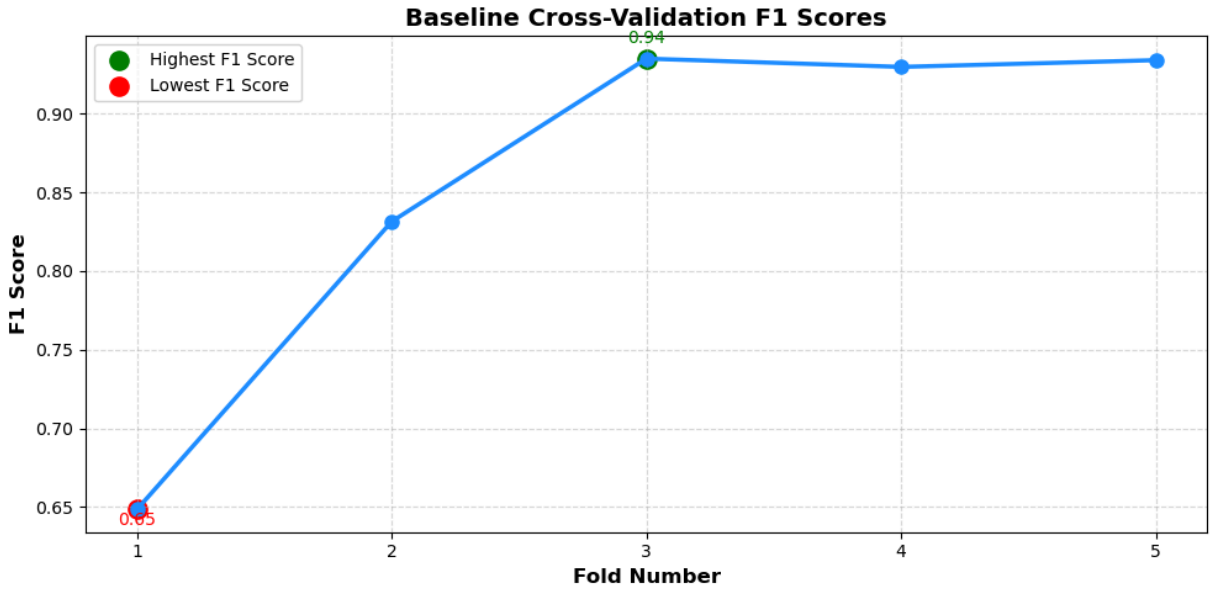
```python
# Evaluate baseline Random Forest using resampled data and cross-validation
cv_scores = cross_val_score(
    RandomForestClassifier(random_state=42),
    X_train_resampled, y_train_resampled, cv=5, scoring="f1_weighted"
)
print(f"Baseline Cross-validation F1 Scores: {cv_scores}")
print(f"Mean Baseline F1 Score: {cv_scores.mean():.4f}")
```

```
Baseline Cross-validation F1 Scores: [0.64845926 0.83146245 0.93501335 0.92976711 0.
93403883]
Mean Baseline F1 Score: 0.8557
```

Now, let's visualize the results to gain a complete understanding of the baseline scores. The model might overfit on certain folds, which can lead to fluctuations in the scores. Further refinement, such as using grid search or adjusting feature selection, can help improve stability.

In [101…

```python
def visualize_cross_validation(cv_scores):
    plt.figure(figsize=(10, 5))

    # Plot line
    plt.plot(range(1, len(cv_scores) + 1), cv_scores, marker='o', markersize=8,
             color='dodgerblue', lw=2.5, linestyle='-')

    # Labels and title
    plt.xlabel("Fold Number", fontsize=12, fontweight="bold")
    plt.ylabel("F1 Score", fontsize=12, fontweight="bold")
    plt.title("Baseline Cross-Validation F1 Scores", fontsize=14, fontweight="bold"

    # Ticks and grid
    plt.xticks(range(1, len(cv_scores) + 1), fontsize=10)
    plt.yticks(fontsize=10)
    plt.grid(True, linestyle='--', alpha=0.5)

    # Highlight best and worst scores
    max_idx = np.argmax(cv_scores) + 1
    min_idx = np.argmin(cv_scores) + 1
    plt.scatter(max_idx, max(cv_scores), color="green", s=120, label="Highest F1 Sc
    plt.scatter(min_idx, min(cv_scores), color="red", s=120, label="Lowest F1 Score

    # Optional: annotate the scores
    plt.text(max_idx, max(cv_scores)+0.01, f"{max(cv_scores):.2f}", color="green",
    plt.text(min_idx, min(cv_scores)-0.01, f"{min(cv_scores):.2f}", color="red", fo

    # Legend and layout
    plt.legend(fontsize=10)
    plt.tight_layout()
    plt.show()

visualize_cross_validation(cv_scores)
```



The plot indicates that performance varies across folds, with some folds underperforming. This highlights the importance of cross-validation to ensure the model generalizes well. Additionally, hyperparameter tuning may help stabilize performance and reduce variability between folds.

6.2 Hyperparameter Tuning using GridSearchCV

Fine-tuning the model's hyperparameters can significantly enhance predictive performance by better handling underrepresented classes and capturing complex patterns within the data. Incorporating cross-validation during this process helps prevent overfitting, ensuring that the model generalizes effectively to unseen data. Using F1-weighted scoring balances precision and recall, minimizing the likelihood of false predictions. Overall, hyperparameter tuning refines the model's ability to recognize meaningful patterns, resulting in more accurate and reliable accident forecasting.

Collectively, these steps improve the model's capacity to identify accident-prone conditions, thereby strengthening the quality of predictions and enhancing the overall robustness of the forecasting system.

In [102…

```
import traceback

def hyperparameter_tuning(X_train_resampled, y_train_resampled):
    """
    Performs hyperparameter tuning using GridSearchCV on resampled training data
    and returns the best estimator.
    """
    param_grid = {
        'n_estimators': [100, 200],
        'max_depth': [10, 20, None],
```

```
        'min_samples_split': [2, 5],
        'min_samples_leaf': [1, 2],
        'class_weight': [None, 'balanced']
    }

    grid_search = GridSearchCV(
        estimator=RandomForestClassifier(random_state=42),
        param_grid=param_grid,
        scoring='f1_weighted',
        cv=5,
        n_jobs=-1,
        verbose=1,
        refit=True,
        error_score='raise'
    )

    try:
        grid_search.fit(X_train_resampled, y_train_resampled)
        print("Best Parameters:", grid_search.best_params_)
        return grid_search.best_estimator_
    except Exception as e:
        print("GridSearchCV failed:", str(e))
        traceback.print_exc()
        return None

# Perform hyperparameter tuning
best_model = hyperparameter_tuning(X_train_resampled, y_train_resampled)
```

```
Fitting 5 folds for each of 48 candidates, totalling 240 fits
Best Parameters: {'class_weight': None, 'max_depth': None, 'min_samples_leaf': 1, 'm
in_samples_split': 5, 'n_estimators': 200}
```

In [103…

```
# Train the optimized Random Forest model
best_model.fit(X_train_resampled, y_train_resampled)
```

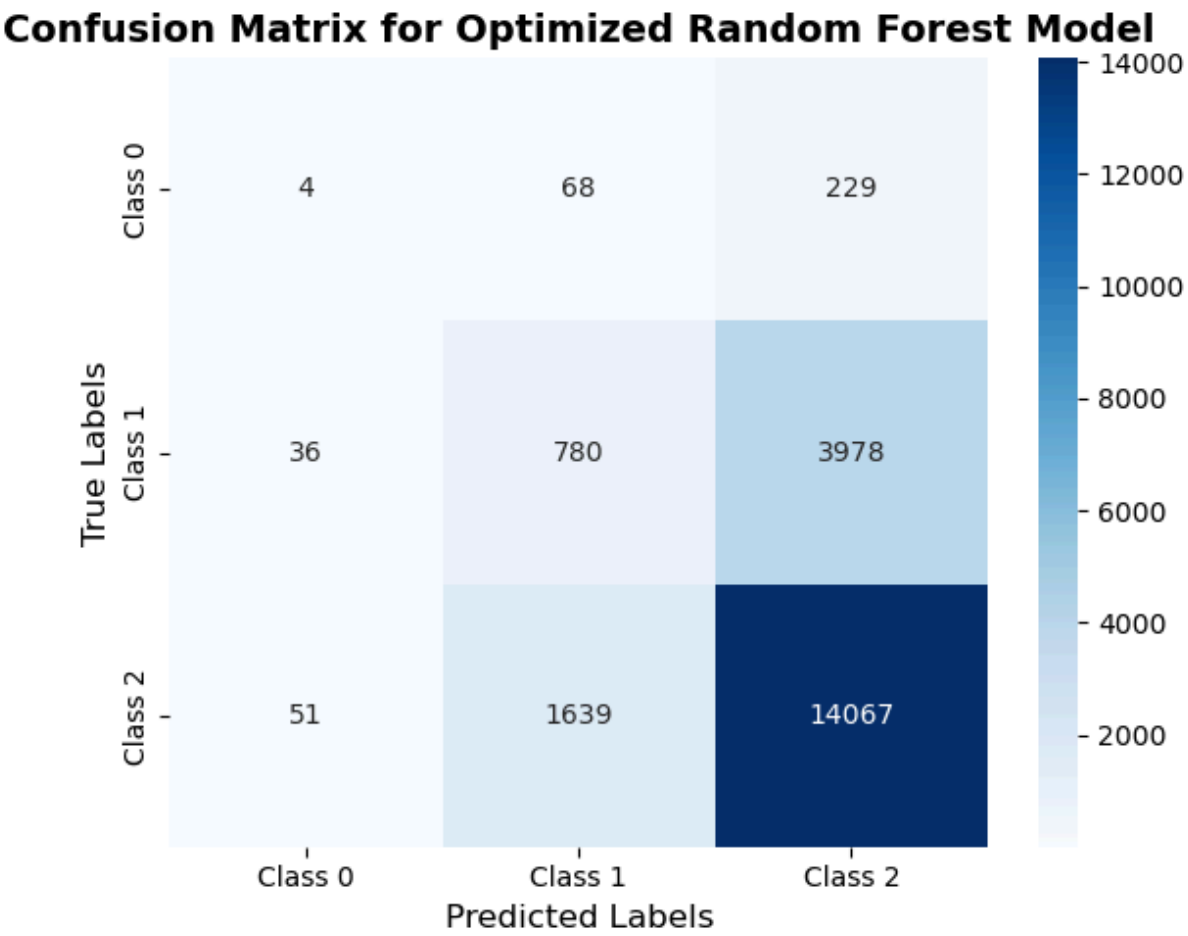
Out[103…

RandomForestClassifier

```
RandomForestClassifier(min_samples_split=5, n_estimators=200, random_state
=42)
```

In [105…

```
# Compute confusion matrix
cm = confusion_matrix(y_test, rf_pred)

# Generate class labels dynamically
class_labels = [f"Class {i}" for i in np.unique(y_test)]

# Plot confusion matrix
plt.figure(figsize=(6, 5))
sns.heatmap(cm, annot=True, fmt='d', cmap="Blues", xticklabels=class_labels, yticklabels=class_labels)
plt.xlabel("Predicted Labels", fontsize=12)
plt.ylabel("True Labels", fontsize=12)
plt.title("Confusion Matrix for Optimized Random Forest Model", fontsize=14, fontweight='bold')
plt.tight_layout()
plt.show()
```



The confusion matrix reveals 63.26% overall accuracy, with strong performance on severe accidents (Class 2).

To further improve the model performance, we will carry out the step below. This could help address class imbalance. Additionally, **n_estimators=100** ensures the model has a reasonable number of trees for stable predictions without excessive computational cost.

In [106…

```
# Train the final model with optimal max_depth
final_rf_model = RandomForestClassifier(
    random_state=42,
    n_estimators=100,
    max_depth=20,
    class_weight="balanced"
)
final_rf_model.fit(X_train_resampled, y_train_resampled)
```

Out[106…

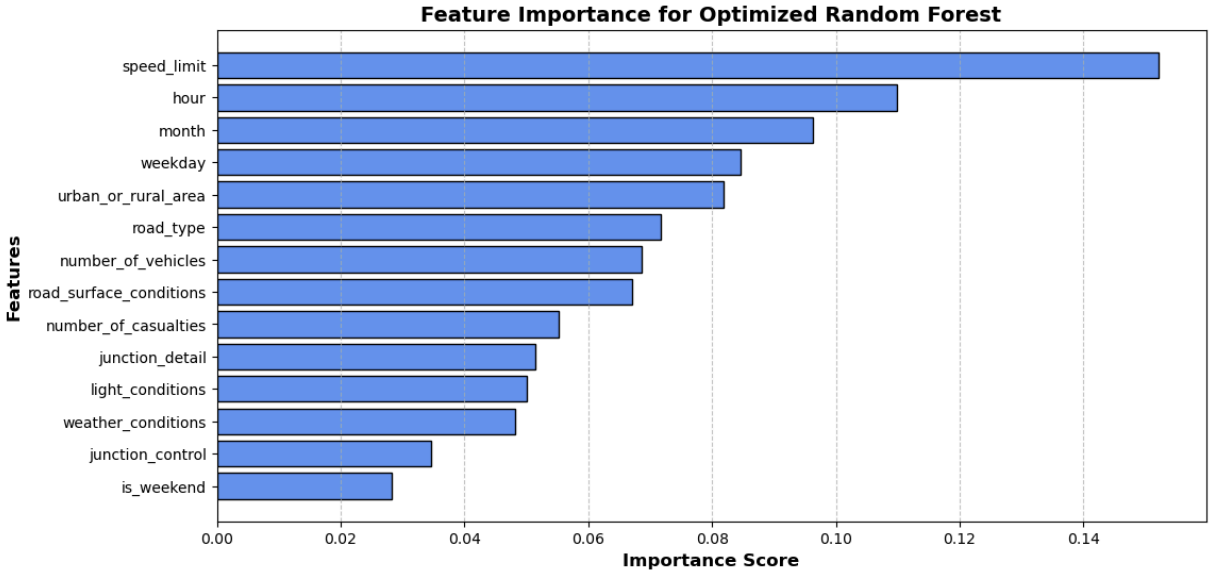
RandomForestClassifier

```
RandomForestClassifier(class_weight='balanced', max_depth=20, random_state
=42)
```

In [107…

```
# Feature importance values
importance = final_rf_model.feature_importances_
```

```
# Sort features by importance
sorted_indices = np.argsort(importance)[::-1]


plt.figure(figsize=(12, 6))
plt.barh(np.array(X_train_resampled.columns)[sorted_indices], importance[sorted_ind

plt.xlabel("Importance Score", fontsize=12, fontweight='bold')
plt.ylabel("Features", fontsize=12, fontweight='bold')
plt.title("Feature Importance for Optimized Random Forest", fontsize=14, fontweight
plt.gca().invert_yaxis()  # Flip to show most important feature on top
plt.grid(axis="x", linestyle="--", alpha=0.7)

# Display the plot
plt.show()
```



The feature importance plot provides insights into the factors that are most influential in predicting accident severity. Indicating features like hour, month, speed limit, and weekday to be the most significant while weekend, urban, rural areas, and road surface condition are less significant.

In [118…

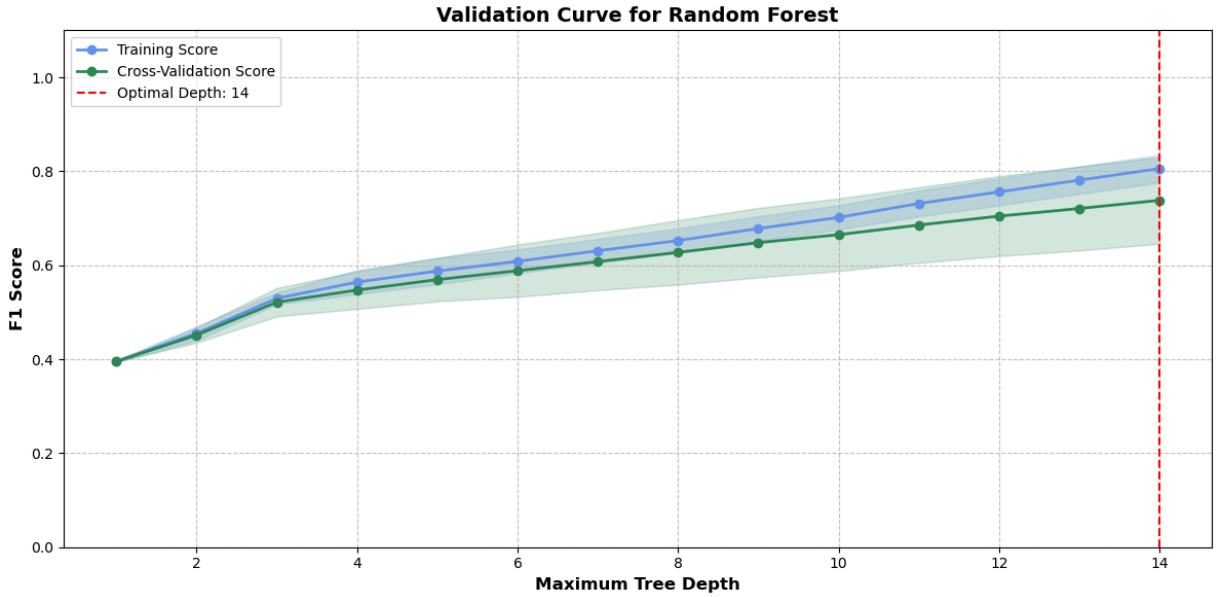
```
import numpy as np
from sklearn.model_selection import validation_curve
from sklearn.ensemble import RandomForestClassifier

# Define the range of the hyperparameter you want to explore
param_range = np.arange(1, 15)  # example for max_depth

# Compute the validation curve
train_scores, valid_scores = validation_curve(
    RandomForestClassifier(random_state=42, n_estimators=100, class_weight='balance
    X_train_resampled,
    y_train_resampled,
    param_name="max_depth",
    param_range=param_range,
    scoring="f1_weighted",
```

```
    cv=3,  # fewer folds than full grid search for speed
    n_jobs=-1
)

# Compute mean and std for plotting
train_scores_mean = np.mean(train_scores, axis=1)
train_scores_std = np.std(train_scores, axis=1)
valid_scores_mean = np.mean(valid_scores, axis=1)
valid_scores_std = np.std(valid_scores, axis=1)
```

In [122…

```
# Step 4: Plot validation curve
plt.figure(figsize=(12, 6))
plt.title("Validation Curve for Random Forest", fontsize=14, fontweight='bold')
plt.xlabel("Maximum Tree Depth", fontsize=12, fontweight='bold')
plt.ylabel("F1 Score", fontsize=12, fontweight='bold')
plt.ylim(0.0, 1.1)
lw = 2

plt.plot(param_range, train_scores_mean, label="Training Score", color="cornflowerb
plt.fill_between(param_range, train_scores_mean - train_scores_std, train_scores_me

plt.plot(param_range, valid_scores_mean, label="Cross-Validation Score", color="sea
plt.fill_between(param_range, valid_scores_mean - valid_scores_std, valid_scores_me

# Highlight optimal parameter
optimal_param = param_range[np.argmax(valid_scores_mean)]
plt.axvline(optimal_param, color='red', linestyle='--', label=f"Optimal Depth: {opt

plt.legend(loc="best", fontsize=10)
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()

# Save + Show
plt.savefig('validation_curve.png')
print(f"Validation curve saved as 'validation_curve.png' in {os.getcwd()}")
plt.show()
```

```
Validation curve saved as 'validation_curve.png' in c:\Users\Idris\OneDrive - Univer
sity College London\Documents\CASA 24_25\TERM 2\Data Science for Spatial Systems\Ass
esement\Accident_Prediction
```



After tuning, the model achieves better fitting at depth 20; it also balances bias and variance, ensuring optimal predictive accuracy for accident occurrence while avoiding overfitting.

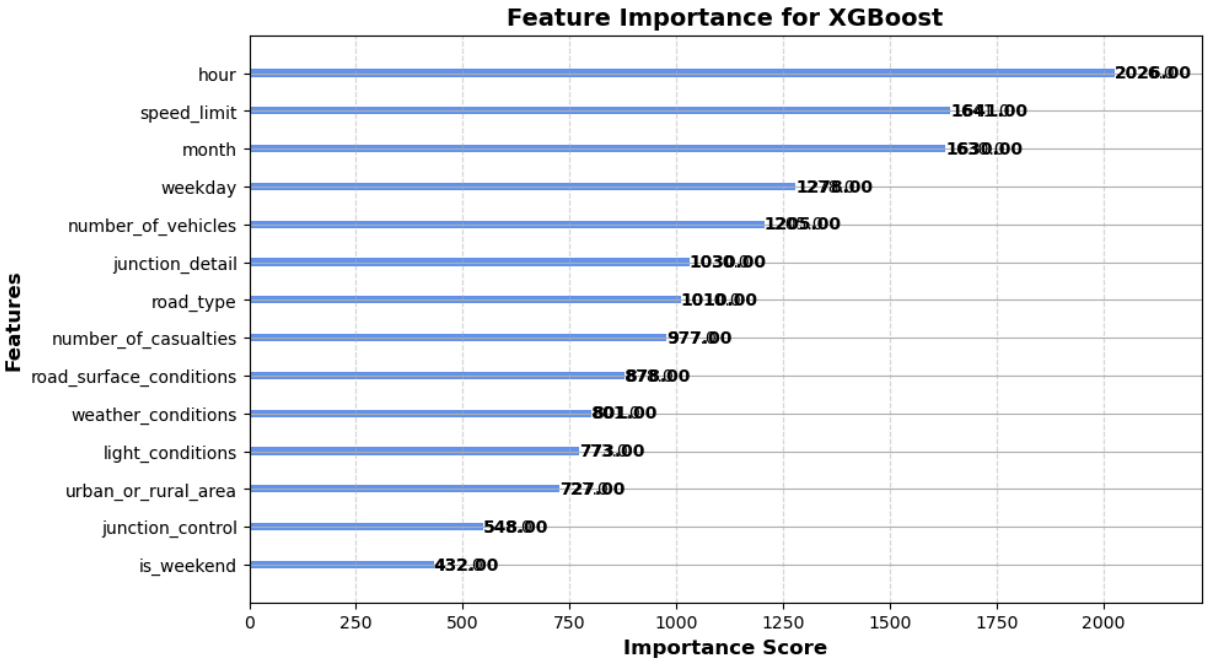
In [ ]:
```
# Encode categorical columns using training set mapping
categorical_cols = X_train_resampled.select_dtypes(include='object').columns
for col in categorical_cols:
    encoder = LabelEncoder()
    X_train_resampled[col] = encoder.fit_transform(X_train_resampled[col].astype(st
    X_test[col] = encoder.transform(X_test[col].astype(str))

# Prepare DMatrix
dtrain = xgb.DMatrix(X_train_resampled, label=y_train_resampled)
dtest = xgb.DMatrix(X_test, label=y_test)

# XGBoost parameters
params = {
    'objective': 'multi:softmax',
    'num_class': len(np.unique(y_train_resampled)),
    'max_depth': 6,
    'eta': 0.1,
    'eval_metric': 'mlogloss',
    'seed': 42
}

# Train model
xgb_model = xgb.train(
    params=params,
    dtrain=dtrain,
    num_boost_round=100,
    evals=[(dtrain, 'train'), (dtest, 'test')],
    early_stopping_rounds=10,
    verbose_eval=10
)

# Predictions
y_pred = xgb_model.predict(dtest).astype(int)
```

```python

# Evaluation
print("\nConfusion Matrix:\n", confusion_matrix(y_test, y_pred))
print("\nClassification Report:\n", classification_report(y_test, y_pred))

# Feature importance
fig, ax = plt.subplots(figsize=(10, 6))
xgb.plot_importance(xgb_model, ax=ax, importance_type="gain", color="cornflowerblue
ax.set_title("Feature Importance for XGBoost", fontsize=14, fontweight="bold")
ax.set_xlabel("Importance Score", fontsize=12, fontweight="bold")
ax.set_ylabel("Features", fontsize=12, fontweight="bold")
ax.grid(axis="x", linestyle="--", alpha=0.5)

# Annotate bars
for patch in ax.patches:
    value = patch.get_width()
    if value > 0:
        ax.text(value + 0.02, patch.get_y() + patch.get_height()/2, f"{value:.2f}",
                fontsize=10, color="black", fontweight="bold", ha="left", va="cente

plt.tight_layout()
plt.show()
```

```
Data types after encoding:
 hour                        float64
weekday                       int32
month                         int32
is_weekend                     bool
weather_conditions            int64
light_conditions              int64
road_surface_conditions       int64
road_type                     int64
junction_detail               int64
junction_control              int64
speed_limit                   int64
urban_or_rural_area           int64
number_of_vehicles            int64
number_of_casualties          int64
dtype: object
[0]     train-mlogloss:1.06490  test-mlogloss:1.06652
[1]     train-mlogloss:1.03553  test-mlogloss:1.03850
[2]     train-mlogloss:1.00960  test-mlogloss:1.01335
[3]     train-mlogloss:0.98711  test-mlogloss:0.99191
[4]     train-mlogloss:0.96648  test-mlogloss:0.97219
[5]     train-mlogloss:0.94857  test-mlogloss:0.95516
[6]     train-mlogloss:0.93189  test-mlogloss:0.93904
[7]     train-mlogloss:0.91671  test-mlogloss:0.92504
[8]     train-mlogloss:0.90228  test-mlogloss:0.91143
[9]     train-mlogloss:0.88961  test-mlogloss:0.89991
[10]    train-mlogloss:0.87817  test-mlogloss:0.88929
[11]    train-mlogloss:0.86719  test-mlogloss:0.87890
[12]    train-mlogloss:0.85716  test-mlogloss:0.87024
[13]    train-mlogloss:0.84685  test-mlogloss:0.86081
[14]    train-mlogloss:0.83688  test-mlogloss:0.85250
[15]    train-mlogloss:0.82788  test-mlogloss:0.84398
[16]    train-mlogloss:0.81893  test-mlogloss:0.83619
[17]    train-mlogloss:0.81062  test-mlogloss:0.82896
[18]    train-mlogloss:0.80313  test-mlogloss:0.82187
[19]    train-mlogloss:0.79570  test-mlogloss:0.81513
[20]    train-mlogloss:0.78805  test-mlogloss:0.80857
[21]    train-mlogloss:0.78160  test-mlogloss:0.80315
[22]    train-mlogloss:0.77480  test-mlogloss:0.79710
[23]    train-mlogloss:0.76872  test-mlogloss:0.79193
[24]    train-mlogloss:0.76271  test-mlogloss:0.78670
[25]    train-mlogloss:0.75730  test-mlogloss:0.78239
[26]    train-mlogloss:0.75227  test-mlogloss:0.77800
[27]    train-mlogloss:0.74755  test-mlogloss:0.77369
[28]    train-mlogloss:0.74258  test-mlogloss:0.76944
[29]    train-mlogloss:0.73842  test-mlogloss:0.76603
[30]    train-mlogloss:0.73402  test-mlogloss:0.76221
[31]    train-mlogloss:0.73004  test-mlogloss:0.75900
[32]    train-mlogloss:0.72603  test-mlogloss:0.75622
[33]    train-mlogloss:0.72153  test-mlogloss:0.75246
[34]    train-mlogloss:0.71769  test-mlogloss:0.74930
[35]    train-mlogloss:0.71354  test-mlogloss:0.74570
[36]    train-mlogloss:0.71000  test-mlogloss:0.74327
[37]    train-mlogloss:0.70625  test-mlogloss:0.74007
[38]    train-mlogloss:0.70265  test-mlogloss:0.73706
[39]    train-mlogloss:0.69982  test-mlogloss:0.73521
```

```
[40]    train-mlogloss:0.69620  test-mlogloss:0.73260
[41]    train-mlogloss:0.69274  test-mlogloss:0.72973
[42]    train-mlogloss:0.68845  test-mlogloss:0.72603
[43]    train-mlogloss:0.68523  test-mlogloss:0.72351
[44]    train-mlogloss:0.68216  test-mlogloss:0.72108
[45]    train-mlogloss:0.67970  test-mlogloss:0.71921
[46]    train-mlogloss:0.67663  test-mlogloss:0.71743
[47]    train-mlogloss:0.67437  test-mlogloss:0.71565
[48]    train-mlogloss:0.67140  test-mlogloss:0.71349
[49]    train-mlogloss:0.66839  test-mlogloss:0.71136
[50]    train-mlogloss:0.66547  test-mlogloss:0.70913
[51]    train-mlogloss:0.66290  test-mlogloss:0.70726
[52]    train-mlogloss:0.66046  test-mlogloss:0.70523
[53]    train-mlogloss:0.65782  test-mlogloss:0.70357
[54]    train-mlogloss:0.65547  test-mlogloss:0.70223
[55]    train-mlogloss:0.65329  test-mlogloss:0.70054
[56]    train-mlogloss:0.65043  test-mlogloss:0.69846
[57]    train-mlogloss:0.64813  test-mlogloss:0.69690
[58]    train-mlogloss:0.64575  test-mlogloss:0.69540
[59]    train-mlogloss:0.64286  test-mlogloss:0.69376
[60]    train-mlogloss:0.64091  test-mlogloss:0.69243
[61]    train-mlogloss:0.63849  test-mlogloss:0.69084
[62]    train-mlogloss:0.63702  test-mlogloss:0.68994
[63]    train-mlogloss:0.63473  test-mlogloss:0.68848
[64]    train-mlogloss:0.63202  test-mlogloss:0.68661
[65]    train-mlogloss:0.62994  test-mlogloss:0.68528
[66]    train-mlogloss:0.62810  test-mlogloss:0.68430
[67]    train-mlogloss:0.62555  test-mlogloss:0.68241
[68]    train-mlogloss:0.62320  test-mlogloss:0.68054
[69]    train-mlogloss:0.62164  test-mlogloss:0.67964
[70]    train-mlogloss:0.61943  test-mlogloss:0.67793
[71]    train-mlogloss:0.61768  test-mlogloss:0.67705
[72]    train-mlogloss:0.61626  test-mlogloss:0.67649
[73]    train-mlogloss:0.61464  test-mlogloss:0.67570
[74]    train-mlogloss:0.61278  test-mlogloss:0.67433
[75]    train-mlogloss:0.61017  test-mlogloss:0.67249
[76]    train-mlogloss:0.60841  test-mlogloss:0.67138
[77]    train-mlogloss:0.60693  test-mlogloss:0.67030
[78]    train-mlogloss:0.60517  test-mlogloss:0.66914
[79]    train-mlogloss:0.60287  test-mlogloss:0.66755
[80]    train-mlogloss:0.60116  test-mlogloss:0.66666
[81]    train-mlogloss:0.59972  test-mlogloss:0.66602
[82]    train-mlogloss:0.59806  test-mlogloss:0.66487
[83]    train-mlogloss:0.59679  test-mlogloss:0.66422
[84]    train-mlogloss:0.59506  test-mlogloss:0.66321
[85]    train-mlogloss:0.59356  test-mlogloss:0.66234
[86]    train-mlogloss:0.59222  test-mlogloss:0.66148
[87]    train-mlogloss:0.59079  test-mlogloss:0.66096
[88]    train-mlogloss:0.58911  test-mlogloss:0.65991
[89]    train-mlogloss:0.58660  test-mlogloss:0.65823
[90]    train-mlogloss:0.58480  test-mlogloss:0.65724
[91]    train-mlogloss:0.58327  test-mlogloss:0.65634
[92]    train-mlogloss:0.58190  test-mlogloss:0.65557
[93]    train-mlogloss:0.58081  test-mlogloss:0.65490
[94]    train-mlogloss:0.57941  test-mlogloss:0.65418
[95]    train-mlogloss:0.57751  test-mlogloss:0.65309
```

```
[96]    train-mlogloss:0.57549  test-mlogloss:0.65177
[97]    train-mlogloss:0.57436  test-mlogloss:0.65129
[98]    train-mlogloss:0.57332  test-mlogloss:0.65057
[99]    train-mlogloss:0.57130  test-mlogloss:0.64939

Confusion Matrix:
 [[   18    41   242]
 [   90   520  4184]
 [   98   945 14714]]

Classification Report:
              precision    recall  f1-score   support

           0       0.09      0.06      0.07       301
           1       0.35      0.11      0.17      4794
           2       0.77      0.93      0.84     15757

    accuracy                           0.73     20852
   macro avg       0.40      0.37      0.36     20852
weighted avg       0.66      0.73      0.68     20852
```

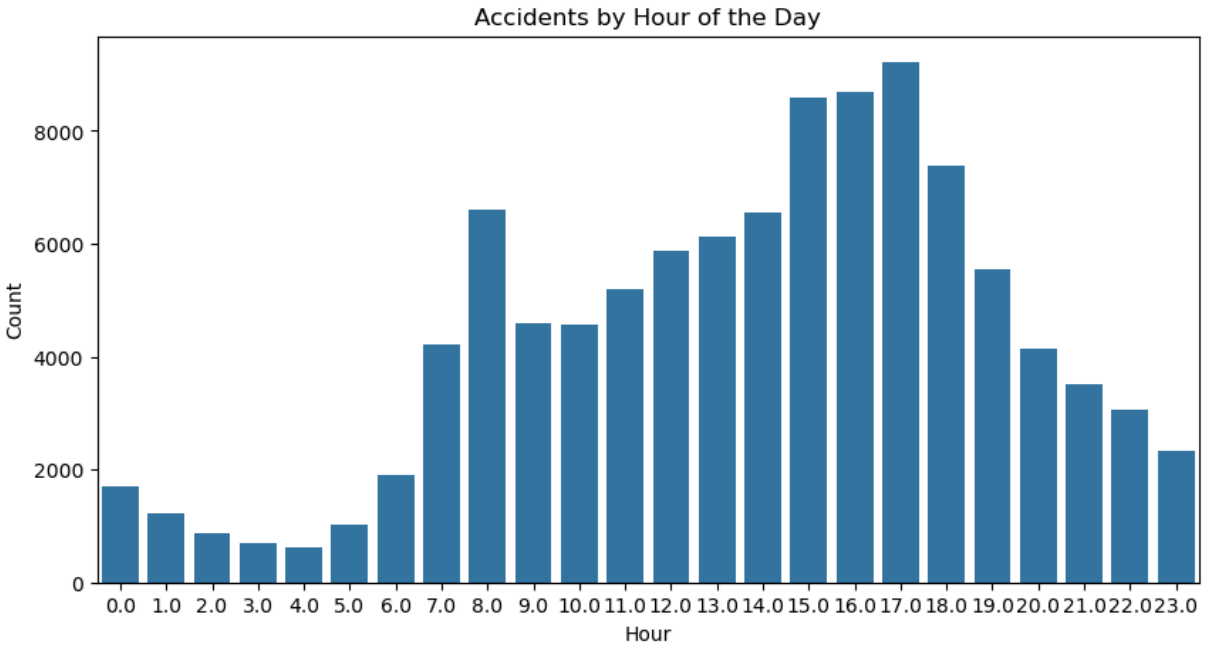


6.3 Key observations on both random forest and XGboost models on road accident prediction

- Time-related features, such as the hour of the day, are highly influential in accident prediction, suggesting that timing and seasonal variations play an important role in London accident prediction. Further exploratory data analysis will explore peak accident hours. The number of vehicles also indicates higher risks during rush hours due to dense traffic, and road characteristics and weather highlight environmental influences on accident likelihood.

In [128…

```
# Accidents per hour
plt.figure(figsize=(10, 5))
sns.countplot(x='hour', data=X, order=sorted(X['hour'].dropna().unique()))
plt.title("Accidents by Hour of the Day")
plt.xlabel("Hour")
plt.ylabel("Count")
plt.show()
```

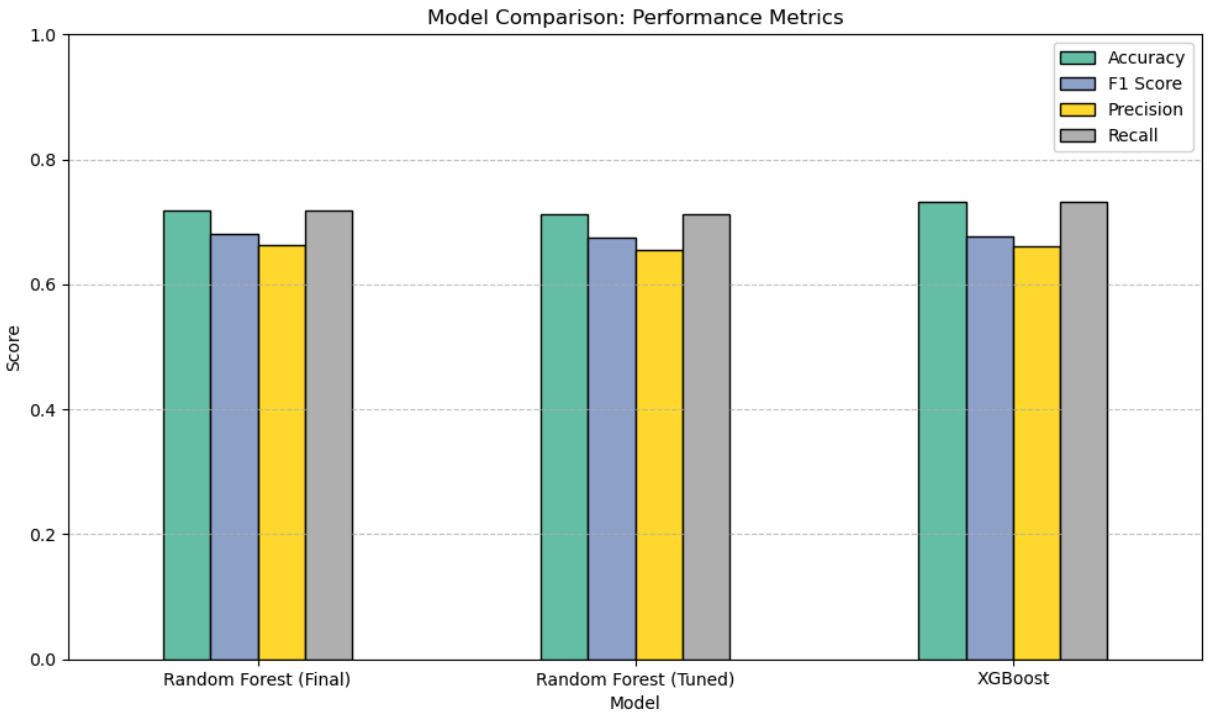


A deeper examination of accident occurrences by hour of the day reveals clear temporal patterns. Road incidents are minimal during the early morning hours when traffic volumes are lowest, as most residents are either asleep or not yet commuting. However, a sharp increase in accidents is observed around 8:00 AM, coinciding with the morning rush hour as Londoners begin their daily routines.

A second notable spike occurs around 3:00 PM, likely reflecting increased traffic from school dismissals and afternoon work commutes. These observations suggest that periods of high traffic congestion are strongly associated with elevated accident rates. Following these peak periods, the frequency of accidents gradually declines until midnight.

In [124…

```
def plot_model_comparison(df_results):
    df_results.set_index("Model")[["Accuracy", "F1 Score", "Precision", "Recall"]].
        kind="bar", figsize=(10, 6), ylim=(0, 1.0), colormap="Set2", edgecolor="bla
    )
    plt.title("Model Comparison: Performance Metrics")
    plt.ylabel("Score")
    plt.xticks(rotation=0)
    plt.grid(axis='y', linestyle='--', alpha=0.7)
    plt.tight_layout()
    plt.show()

# 👇 Call the function here
plot_model_comparison(df_results)
```



In [125…
```
print(df_results)
```

```
                  Model  Accuracy  F1 Score  Precision    Recall
0  Random Forest (Final)  0.718109  0.680242   0.663605  0.718109
1  Random Forest (Tuned)  0.712210  0.674269   0.654433  0.712210
2               XGBoost  0.731441  0.676211   0.661562  0.731441
```

The comparison of the models reveals subtle but important trade-offs between Random Forest and XGBoost. While XGBoost achieved the highest accuracy (0.731) and recall (0.731), indicating that it was more effective at correctly identifying a greater proportion of true accident cases, the Random Forest (Final) model produced the strongest F1 score (0.680), suggesting a slightly better balance between precision and recall. This difference highlights the models' respective strengths: XGBoost generalizes more effectively, reducing the risk of missing accident-prone conditions (false negatives), whereas Random Forest (Final) makes more reliable positive predictions, with fewer false alarms (false positives). Although the differences are relatively small, the choice of model ultimately depends on the application context. In safety-critical scenarios, such as accident prediction where missing true cases could have severe consequences, XGBoost may be preferable due to its higher recall and overall accuracy. However, if the focus is on minimizing unnecessary warnings or interventions, Random Forest (Final) offers a marginally better balance. Thus, the results suggest that while XGBoost demonstrates stronger generalization, Random Forest remains competitive when precision is prioritized.

7.0 Conclusion

[go back to the top]

The findings of this study demonstrate the potential of machine learning for predicting and visualizing road accidents in London. By leveraging supervised models such as Random Forest and XGBoost, the study highlights how predictive analytics can effectively capture the influence of environmental, temporal, and infrastructural factors on accident occurrence. Feature importance analysis further reveals that variables such as time of day, season, and traffic volume exert significant influence, underscoring their role in shaping accident risk.

The comparative evaluation shows that while a tuned Random Forest offers reliable performance, XGBoost demonstrated slightly stronger generalization in the initial trials. This suggests that ensemble learning methods can provide complementary strengths, with their effectiveness depending on the balance between precision and recall required in real-world applications.

Overall, this study contributes to the growing body of research on big data and road safety, offering actionable insights for policymakers, urban planners, and transportation authorities. Integrating predictive modelling into traffic management strategies could enable more targeted interventions, reduce accident risks, and improve public safety outcomes. Nonetheless, limitations such as reliance on historical datasets, potential reporting biases, and the exclusion of real-time contextual data should be addressed in future work. Expanding the approach to incorporate spatial clustering, real-time sensor data, and advanced deep learning models could further enhance predictive accuracy and practical applicability in dynamic urban environments.

8.0 References

[go back to the top]

- **Anjuman, T., Hasanat-E-Rabbi, S., Siddiqui, C.K.A. & Hoque, M. (n.d.) 'Road traffic accident: A leading cause of the global burden of public health injuries and fatalities'.**
- **Beshah, T., Dedefa, D.E. & Abraham, A. (2011) 'Pattern recognition and knowledge discovery from road traffic accident data in Ethiopia: Implications for improving road safety', *Proceedings of the 2011 World Congress on Information and Communication Technologies (WICT 2011)*, IEEE. doi:10.1109/WICT.2011.6141426.**
- **Khosravi, Y., Hosseinali, F. & Adresi, M. (2024) 'Identifying accident prone areas and factors influencing the severity of crashes using machine learning and spatial analyses', *Scientific Reports*, 14, 29836. doi:10.1038/s41598-024-81121-7.**
- **Mitchell, T.M. (1999) 'Machine learning and data mining', *Communications of the ACM*, 42(11), pp.30–36.**

- **Morales, E.F. & Escalante, H.J. (2022) 'A brief introduction to supervised, unsupervised, and reinforcement learning', in *Biosignal Processing and Classification Using Computational Learning and Intelligence*. Elsevier, pp.111–129. doi:10.1016/B978-0-12-820125-1.00017-8.**
- **Pourroostaei Ardakani, S., Liang, X., Mengistu, K.T., So, R.S., Wei, X., He, B. & Cheshmehzangi, A. (2023) 'Road car accident prediction using a machine-learning-enabled data analysis', *Sustainability*, 15, 5939. doi:10.3390/su15075939.**
- **Takale, D.G., Gunjal, S.D., Khan, V.N., Raj, A. & Gujar, S.N. (2022) 'Road accident prediction model using data mining techniques', 20. doi:10.48047/NQ.2022.20.16.NQ880299.**
- **Venkat, A., Gokulnath, M., Guru Vijey, K.P., Thomas, I.S. & Ranjani, D.D. (2020) 'Machine learning based analysis for road accident prediction', *IJETIE*, 6(2), February 2020. Available at: https://ssrn.com/abstract=3550462 (Accessed: [*8th August, 2025]).**
- **Yannis, G., Dragomanovits, A., Laiou, A., La Torre, F., Domenichini, L., Richter, T., Ruhl, S., Graham, D. & Karathodorou, N. (2017) 'Road traffic accident prediction modelling: a literature review', *Proceedings of the Institution of Civil Engineers - Transport*, 170, pp.245–254. doi:10.1680/jtran.16.00067.**
- **Yassin, S.S. & Pooja (2020) 'Road accident prediction and model interpretation using a hybrid K-means and random forest algorithm approach', *SN Applied Sciences*, 2, 1576. doi:10.1007/s42452-020-3125-1.**